7.1 — Périmètre du MVP

Fonctionnalités incluses :

- ✅ Upload CV (PDF) + extraction automatique via LLM
- ✅ Formulaire de préférences : secteur, type de contrat, localisation, mots-clés
- ✅ Recherche d'entreprises et d'offres déléguée au LLM (pas de scraping — voir faisabilité juridique)
- ✅ Analyse de correspondance profil/offre via LLM avec score de matching
- ✅ Génération de lettre de motivation personnalisée par entreprise
- ✅ Notification mail avant postulation (confirmation manuelle obligatoire)
- ✅ Tableau de bord de suivi des candidatures avec gestion des statuts et relances suggérées

Fonctionnalités hors MVP (V2+) :

- ❌ Adaptation dynamique du CV à chaque offre
- ❌ Scraping réel de plateformes
- ❌ Soumission automatique de candidatures
- ❌ Annuaire communautaire
- ❌ Mise en relation entre étudiants
- ❌ Aide à la création de CV from scratch

7.2 — Architecture technique

- **Frontend** : React (Vite) — maîtrisé, léger, SPA suffisant pour le MVP
- **Backend** : Node.js / Express — cohérence JS full-stack, API REST simple
- **Base de données** : SQLite en développement, PostgreSQL (Neon) en production
- **LLM** : API Anthropic Claude Sonnet — meilleure qualité de génération de texte long
- **Notifications** : Resend (mail) — tier gratuit généreux, API simple
- **Hébergement** : Railway (backend + BDD) + Vercel (frontend)
- **Authentification** : JWT stocké en cookie httpOnly

7.3 — User Flow MVP

1. Création de compte (email + mot de passe)
2. Upload CV PDF → extraction LLM → profil pré-rempli à valider
3. Formulaire préférences → secteur, contrat, ville, mots-clés
4. Lancement recherche → LLM génère liste d'entreprises + offres cibles
5. Score de matching calculé pour chaque résultat
6. Pour chaque cible > seuil : génération lettre personnalisée
7. Notification mail récap → lien de confirmation
8. Validation manuelle → candidature enregistrée en base
9. Dashboard mis à jour → suivi statuts + suggestion relance à J+7

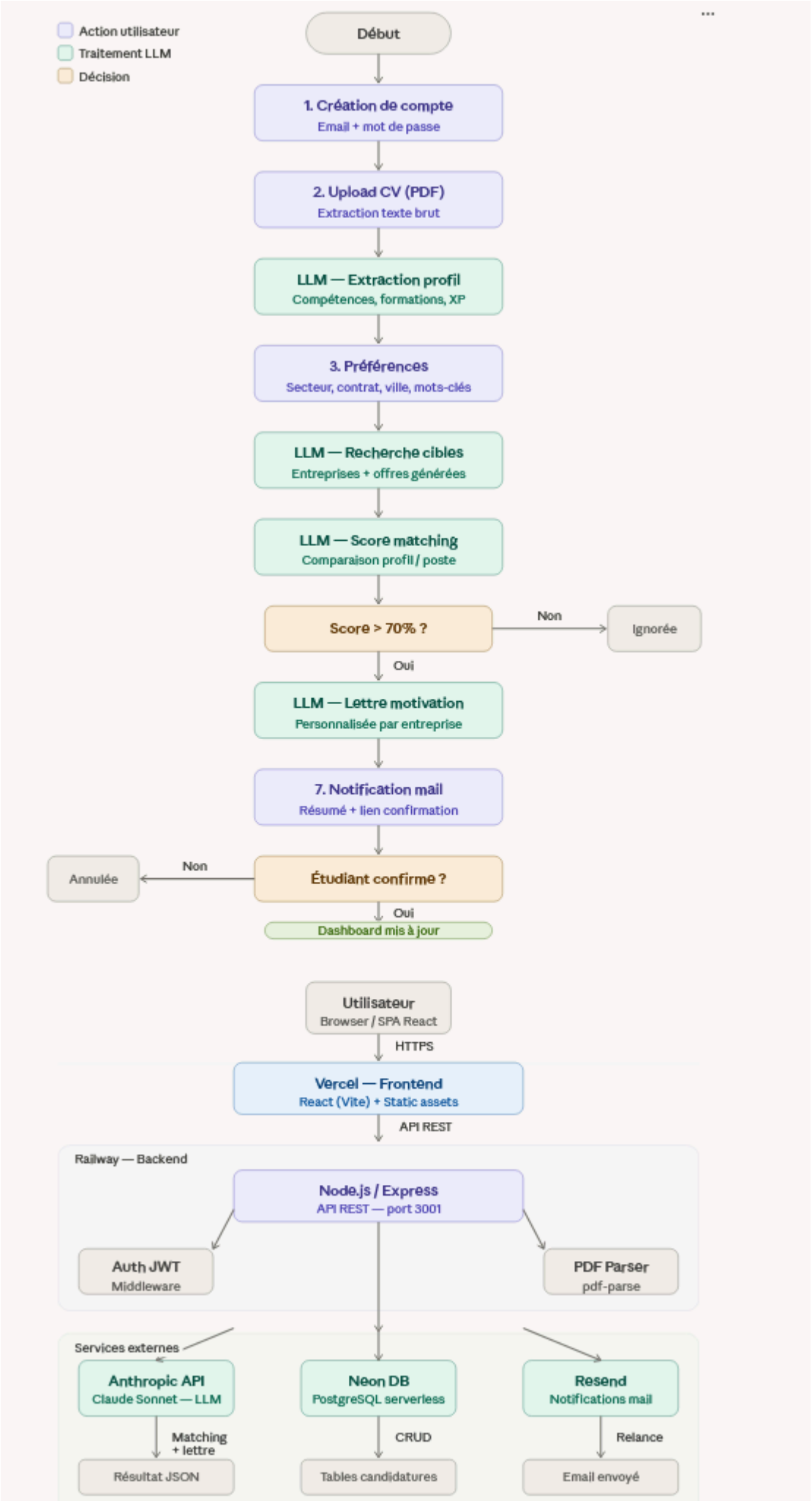
7.4 — RGPD

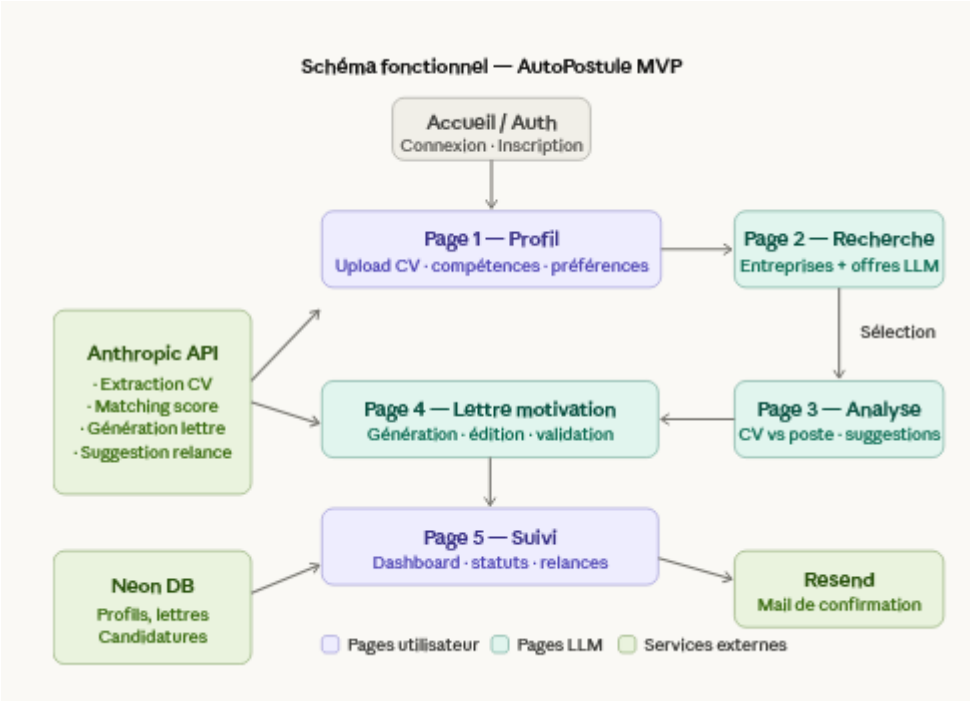
- **Données collectées** : email, CV (texte extrait), préférences, historique candidatures
- **Stockage** : données chiffrées au repos, HTTPS en transit, CV texte uniquement (pas de fichier PDF stocké)
- **Conservation** : données actives tant que le compte existe, supprimées 30 jours après clôture
- **Suppression compte** : bouton "Supprimer mon compte" → suppression immédiate et irréversible de toutes les données

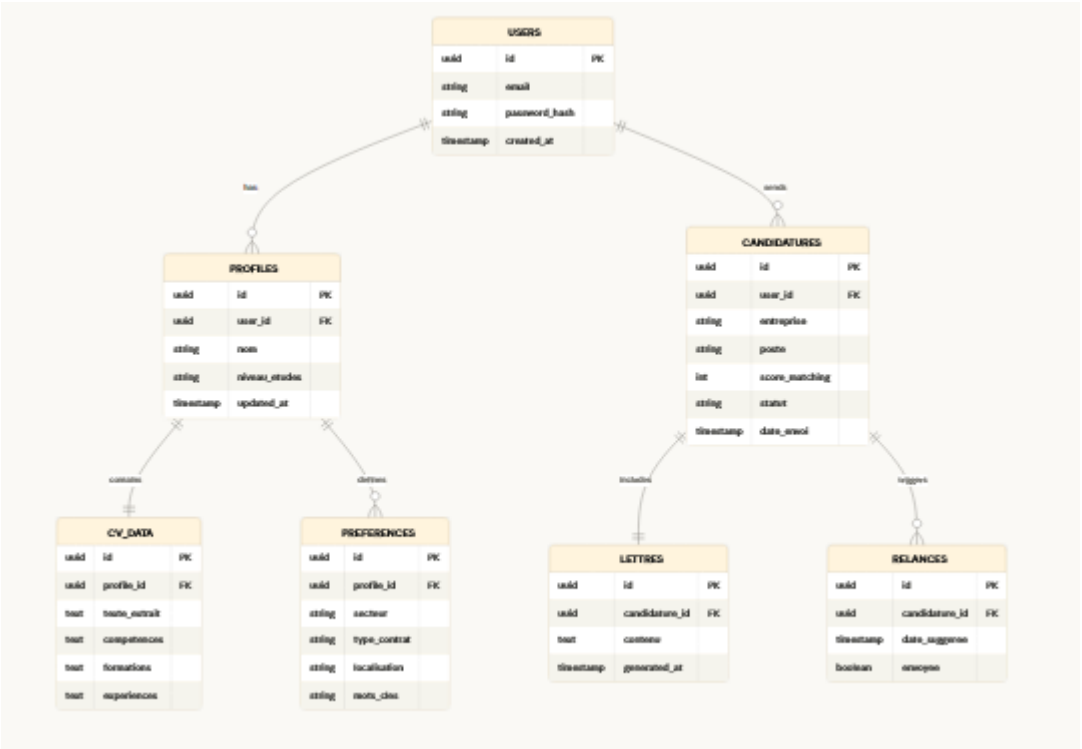
---

7.5 — Critères d'acceptation (Definition of Done)

- ✅ Un utilisateur crée un compte et uploade son CV en moins de 5 minutes
- ✅ Le LLM extrait correctement les infos du CV dans 80% des cas testés
- ✅ L'outil propose au moins 5 entreprises pertinentes selon les préférences
- ✅ Une lettre de motivation distincte est générée pour chaque cible
- ✅ Aucune candidature n'est enregistrée sans validation explicite de l'utilisateur
- ✅ Le dashboard affiche le bon statut pour chaque candidature
- ✅ Une suggestion de relance apparaît après 7 jours sans réponse


Action utilisateur
Traitement LLM
Décision
Début
1. Création de compte
Email + mot de passe
2. Upload CV (PDF)
Extraction texte brut
LLM — Extraction profil
Compétences, formations, XP
3. Préférences
Secteur, contrat, ville, mots-clés
LLM — Recherche cibles
Entreprises + offres générées
LLM — Score matching
Comparaison profil / poste
Score > 70% ?
Non
Ignorée
Oui
LLM — Lettre motivation
Personnalisée par entreprise
7. Notification mail
Résumé + lien confirmation
Étudiant confirme ?
Non
Annulée
Oui
Dashboard mis à jour
Utilisateur
Browser / SPA React
HTTPS
Vercel — Frontend
React (Vite) + Static assets
API REST
Railway — Backend
Node.js / Express
API REST — port 3001
Auth JWT
Middleware
PDF Parser
pdf-parse
Services externes
Anthropic API
Claude Sonnet — LLM
Neon DB
PostgreSQL serverless
Resend
Notifications mail
Matching
+ lettre
CRUD
Relance
Résultat JSON
Tables candidatures
Email envoyé





Phase 0 — Préparation (1-2 jours)

Objectif : Avoir un environnement prêt et ne pas perdre de temps en setup pendant le développement.

Ce que tu fais toi-même :

- Créer un repo GitHub `autopostule`
- Créer un compte Railway, Vercel, Neon DB, Resend, Anthropic (récupérer les clés API)
- Installer Node.js, VS Code, Git si pas déjà fait

Structure de dossiers à créer :

```
autopostule/
├── client/        ← React Vite
├── server/        ← Node.js Express
└── README.md
```

Livrable : Repo GitHub créé, toutes les clés API disponibles dans un fichier `.env.example`, structure de dossiers en place.

Phase 1 — Backend fondation (Semaine 1)

Objectif : Avoir un serveur Express qui tourne, connecté à la base de données, avec l'authentification fonctionnelle.

Étapes dans l'ordre :

1.1 — Initialisation Express Créer le serveur avec les routes de base, middleware CORS, body-parser, gestion des erreurs globale. Tester avec un `GET /health` qui retourne `{ status: "ok" }`.

1.2 — Base de données Connecter Neon DB (PostgreSQL), créer les 7 tables du schéma (USERS, PROFILES, CV_DATA, PREFERENCES, CANDIDATURES, LETTRES, RELANCES). Utiliser des migrations simples en SQL brut ou Prisma si tu veux un ORM.

1.3 — Authentification JWT Routes `POST /auth/register` et `POST /auth/login`. Hash du mot de passe avec bcrypt. Génération et vérification du token JWT. Middleware d'auth à appliquer sur toutes les routes protégées.

Avec Claude Code : donne-lui le schéma de BDD et demande-lui de générer les migrations SQL + le middleware JWT complet.

Livrable : Serveur qui tourne sur `localhost:3001`, authentification testable via Postman ou Insomnia, tables créées en base.

Phase 2 — Frontend fondation (Semaine 1-2)

Objectif : Avoir l'application React avec la navigation entre les 5 pages et l'authentification côté client.

Étapes dans l'ordre :

2.1 — Init React Vite Créer le projet, installer React Router, Axios pour les appels API, Tailwind CSS pour le style.

2.2 — Pages vides avec navigation Créer les 5 pages (Profil, Recherche, Analyse, Lettre, Suivi) + la page Auth. Router configuré avec routes protégées (redirect vers login si pas de token).

2.3 — Formulaire d'auth Pages login et register connectées au backend. Token JWT stocké dans un cookie httpOnly via le backend ou en localStorage pour le MVP. Gestion des erreurs (mauvais mot de passe, email déjà utilisé).

Avec Claude Code : donne-lui la liste des 5 pages et demande-lui de générer la structure React Router complète avec les pages vides et le guard d'authentification.

Livrable : Application navigable sur `localhost:5173`, login/register fonctionnel, redirection automatique selon l'état d'auth.

Phase 3 — Upload CV et extraction LLM (Semaine 2)

Objectif : L'étudiant uploade son PDF, le texte est extrait et envoyé au LLM qui retourne un profil structuré.

Étapes dans l'ordre :

3.1 — Upload fichier côté backend Route `POST /profile/cv` avec Multer pour gérer le fichier. Extraction du texte brut avec `pdf-parse`. Texte stocké dans `CV_DATA`.

3.2 — Appel LLM extraction Envoyer le texte au LLM avec un prompt qui demande de retourner un JSON structuré : compétences, formations, expériences. Parser et stocker le résultat en base.

3.3 — Formulaire préférences Page Profil avec formulaire : secteur, type de contrat, localisation, mots-clés. Sauvegarde dans `PREFERENCES`.

Avec Claude Code : donne-lui la route Multer à créer + le prompt LLM d'extraction à optimiser.

Livrable : Upload d'un PDF → profil structuré affiché dans l'interface, préférences sauvegardées.

Phase 4 — Recherche intelligente (Semaine 3)

Objectif : À partir du profil, le LLM génère une liste d'entreprises et d'offres cibles avec un score de matching.

Étapes dans l'ordre :

4.1 — Route de recherche `POST /search/launch` qui récupère le profil complet + préférences et construit le prompt de recherche.

4.2 — Prompt de recherche Le LLM reçoit le profil et les préférences, retourne un JSON avec 5 entreprises et 5 offres, chacune avec nom, secteur, description, et score de matching sur 100.

4.3 — Affichage résultats Page Recherche affiche les cartes résultats. Clic sur une carte → navigation vers la page Analyse avec l'entreprise sélectionnée en contexte.

Avec Claude Code : donne-lui le format JSON attendu en sortie du LLM et demande-lui de générer le prompt complet + le parsing de la réponse.

Livrable : Bouton "Lancer la recherche" → liste de 5 entreprises + 5 offres avec scores affichés en cartes.

Phase 5 — Analyse et génération lettre (Semaine 4)

Objectif : Pour une entreprise sélectionnée, le LLM analyse le CV, propose des améliorations, et génère une lettre de motivation.

Étapes dans l'ordre :

5.1 — Analyse CV vs poste Route `POST /analyse` avec le CV + la fiche entreprise. LLM retourne deux blocs JSON : suggestions générales (valables pour ce type de poste) et suggestions spécifiques (adaptées à cette entreprise précise).

5.2 — Affichage analyse Page Analyse avec deux colonnes distinctes. Chaque suggestion est une carte cliquable.

5.3 — Génération lettre Route `POST /lettre/generate`. LLM génère la lettre avec les infos entreprise + profil utilisateur. Résultat affiché dans un champ texte éditable. Bouton "Sauvegarder" → stockage en base dans `LETTRES`.

Avec Claude Code : c'est la phase la plus riche en prompt engineering, utilise-le pour itérer sur les prompts jusqu'à obtenir une qualité satisfaisante.

Livrable : Analyse en deux niveaux affichée, lettre de motivation personnalisée générée et éditable, sauvegarde fonctionnelle.

Phase 6 — Suivi et relances (Semaine 5)

Objectif : L'étudiant suit ses candidatures et reçoit des suggestions de relance au bon moment.

Étapes dans l'ordre :

6.1 — Enregistrement candidature Quand l'étudiant valide une lettre → création d'une entrée dans `CANDIDATURES` avec statut "À envoyer". Notification mail via Resend avec résumé de la candidature.

6.2 — Dashboard suivi Page Suivi avec tableau : entreprise, poste, score, statut (À envoyer / Envoyé / Relancé / Répondu / Refusé), date. Changement de statut manuel par clic.

6.3 — Suggestion relance Si statut "Envoyé" depuis plus de 7 jours → afficher une suggestion de message de relance générée par le LLM. L'étudiant peut la copier ou la marquer comme envoyée.

Avec Claude Code : génération du composant tableau de suivi complet + la logique de détection des relances à suggérer.

Livrable : Dashboard fonctionnel, changement de statuts, suggestion de relance après 7 jours, mail de confirmation envoyé via Resend.

Phase 7 — Buffer, tests et déploiement (Semaines 6-7)

Objectif : Corriger les bugs, tester avec de vrais utilisateurs, déployer en ligne.

Étapes dans l'ordre :

7.1 — Tests informels Faire tester par 3-5 camarades avec de vrais CVs. Noter les bugs, les incompréhensions, les lenteurs. Pas de tests automatisés pour le MVP — tests manuels suffisants.

7.2 — Corrections prioritaires Traiter uniquement les bugs bloquants et les problèmes d'UX majeurs. Résister à la tentation d'ajouter des fonctionnalités.

7.3 — Déploiement Backend sur Railway, frontend sur Vercel, base de données sur Neon. Variables d'environnement configurées en production. Tester le flux complet en production avant de partager.

7.4 — Documentation minimale README avec instructions d'installation, description du projet, stack technique. Utile pour le CV et pour montrer le projet à un recruteur.

Livrable final : Application en ligne accessible par URL publique, flux complet fonctionnel de bout en bout, README propre sur GitHub.